საბოლოო ტექნიკური ანგარიში

პროფესიული საგანმანათლებლო პროგრამა: Front-end პროგრამირება · მოდული: პრაქტიკული პროექტი (06.1.1.9)

სტუდენტი	თინათინ მაისურაძე
შესრულების ფორმატი	ინდივიდუალური
პროექტის დასახელება და სფერო	MediBook: ექიმის ონლაინ ჯავშნის სისტემა (ჯანდაცვა)
GitHub რეპოზიტორია	https://github.com/tinatinmaisurradze/medibook
Live Demo	https://medibook-chi-tan.vercel.app
დოკუმენტაცია (Notion)	https://curved-foxtail-971.notion.site/MediBook-4059f635336d809dabb0c6c5c1c2ea5f
პროექტის მართვის დაფა	https://github.com/users/tinatinmaisurradze/projects/2
სატესტო ანგარიშები	პაროლი ყველასთვის: `Demo1234!` · პაციენტი `patient1@medibook.ge` · ექიმი `doctor1@medibook.ge` · ადმინისტრატორი `admin@medibook.ge`

სწავლის შედეგი 1: სამუშაოს დაგეგმვა და ორგანიზება

1.1. სამუშაო გეგმა, ეტაპები (Milestones) და რისკების განსაზღვრა

პროექტის მიზანი და ბიზნეს-ამოცანა. ექიმთან ვიზიტის დაჯავშნა ხშირად ტელეფონით ან რეგისტრატურაში მისვლით ხდება: პაციენტი ვერ ხედავს თავისუფალ დროს, ექიმს და ადმინისტრაციას ერთიანი სურათი არ აქვთ, ორმაგი ჯავშნისა და გაუქმებული ვიზიტის გამოუყენებელი დროის რისკი არსებობს. MediBook ერთ ვებ-აპლიკაციაში აერთიანებს სამ მომხმარებელს: **პაციენტს** (ძებნა, დაჯავშნა, ისტორია), **ექიმს** (განრიგი, პაციენტები, ჩანაწერები) და **კლინიკის ადმინისტრატორს** (გრაფიკები, კაბინეტები, გაუქმებები). თითოეული როლი ხედავს მხოლოდ თავის სივრცეს.

განხორციელების ეტაპები და ვადები

ეტაპი	ვადა	შინაარსი
I: დაგეგმვა და არქიტექტურა	26 ოქტომბერი	პროექტის არჩევა სამი ვარიანტიდან, ტექნიკური დავალება, როლები, რისკების ანალიზი, API და RBAC დაგეგმვა, ტექნოლოგიური სტეკი
II: UI/Layout და საფუძველი	26 ოქტომბერი	Vite + React + TypeScript + Tailwind, დიზაინ-ტოკენები, ფოლდერების სტრუქტურა, ლეიაუთები, mock API, ავტორიზაცია, ძებნა და ჯავშანი, GitHub-ზე ატვირთვა
III: ბიზნეს-ლოგიკა, CRUD და State	27 ოქტომბერი	ვიზიტის ჩანაწერი, პროფილები და სამედიცინო ისტორია, შეფასებები, ექიმის მთავარი გვერდი, კალენდარი და პაციენტები, ადმინის სივრცე, სერვერის მხარის ვალიდაცია და უსაფრთხოების წესები
IV: ოპტიმიზაცია, ტესტირება, დოკუმენტაცია	28-31 ოქტომბერი	დეპლოი Vercel-ზე, ბაგების გასწორება, ლენდინგის გადაკეთება უკუკავშირის შემდეგ, ექიმის გრაფიკი ერთი ქალაქის რამდენიმე კლინიკაში და ფასები, სატესტო გადახდა, ისტორიის ფილტრები და გვერდებად დაყოფა, თვის კალენდარი დაჯავშნისთვის, ქართულ-ინგლისური ინტერფეისი, არასავალდებულო რეალური სერვერი, უსაფრთხოების გადამოწმება, ErrorBoundary და Offline რეჟიმი, QA ანგარიში, README, ანგარიში და წარდგენა

სამუშაოს ვადების ფაქტობრივი აღრიცხვა მოცემულია Notion-ის Timeline ცხრილსა და GitHub Projects დაფაზე.

იდენტიფიცირებული რისკები და მათი პრევენცია

1. **ორმაგი ჯავშანი (Double-Click / Race Condition).** → *პრევენცია:* ჯავშნის მომენტში სერვერი თავიდან ამოწმებს სლოტის თავისუფლებას და აბრუნებს `SLOT_TAKEN`-ს; პაციენტი ერთსა და იმავე დროს ორ ექიმთან ვერ ჩაიწერება (`PATIENT_BUSY`); ღილაკი გაგზავნისას გამოირთვება; ერთი ჩანართის ჯავშანი მეორე ჩანართში გადადის „დაკავებულია“-ზე გადატვირთვის გარეშე (`storage` მოვლენა).
2. **მონაცემების დაკარგვა გვერდის განახლებისას.** → *პრევენცია:* მონაცემთა ბაზა ინახება `localStorage`-ში, სესიის ტოკენი `sessionStorage`-ში, ხოლო განახლებისას ტოკენი მოწმდება ავტომატურად (`AuthContext`).
3. **დროის გადაჭარბება.** → *პრევენცია:* MVP-ის წინასწარ განსაზღვრა (ავტორიზაცია, ძებნა, ჯავშანი), სამუშაოს მცირე ეტაპებად დაყოფა, თითოეული ეტაპი ცალკე კომიტით და უკან დაბრუნების შესაძლებლობით.
4. **ბიბლიოთეკის ან განახლების გამო ვიზუალის დაზიანება.** → *პრევენცია და რეალური შემთხვევა:* დიდმა განახლებამ ვიზუალი დააზიანა, ვერსია Git-ით (`git revert`) დავაბრუნე და მერე ცვლილებები მცირე, ცალკე გატესტილ ეტაპებად გავყავი.
5. **ბარათის მონაცემების რისკი გადახდისას.** → *პრევენცია:* გადახდის გვერდი სატესტო რეჟიმშია და ბანკს არ უკავშირდება; ბარათი მოწმდება (Luhn, ვადა, CVC) და მერე იკარგება, ინახება მხოლოდ ბრენდი და ბოლო 4 ციფრი; 5 წარუმატებელი მცდელობის შემდეგ გადახდა იბლოკება.
6. **უსაფრთხოების რისკი სამედიცინო მონაცემებზე.** → *პრევენცია:* როლებზე დაფუძნებული წვდომა, ადმინისტრატორი სამედიცინო ტექსტს ვერ ხედავს, ექიმი

პაციენტის ბარათს ხსნის მხოლოდ მაშინ, თუ ვიზიტი აქვს, პაციენტი მართავს ისტორიის გაზიარებას.

1.2. რესურსებისა და მეთოდების შერჩევა

მიმართულება	არჩევანი და დასაბუთება
Core და Build	**React 18 + Vite + TypeScript.** კომპონენტური მიდგომა სამი როლისთვის საერთო UI-ის გამოსაყენებლად, სწრაფი ბილდი, ტიპიზაცია შეცდომების აღმოსაჩენად კოდის გაშვებამდე.
Styling და UI	**Tailwind CSS + CSS ცვლადები (Design Tokens).** ერთიანი ფერები, ინტერვალები და ფორმები `:root`-ში, რესპონსიულობა პრეფიქსებით (`sm`, `md`, `lg`).
State და Persistence	**Context API + custom hooks + localStorage.** გლობალური მდგომარეობა მცირეა (მომხმარებელი და შეტყობინებები), ამიტომ Redux ზედმეტია; ბაზა ინახება `localStorage`-ში mock API-ის უკან, ხოლო `services/http.ts`-ის შეცვლა რეალურ სერვერზე გადასვლის საშუალებას იძლევა.
Validation და Forms	**Zod + React Hook Form.** ერთი სქემა ფორმაშიც და სერვერის მხარესაც, ველიდან გასვლისას (fail-fast) შეტყობინებით.
ენები (i18n)	**საკუთარი მსუბუქი მოდული** (`src/i18n`): ყველა ტექსტი ერთი ფუნქციით არის გატარებული, ინგლისური ვერსიები ერთ ფაილშია (`en.ts`), ენა ინახება და გვერდი ახალ ენაზე იტვირთება. ცალკე ბიბლიოთეკა არ დაგვჭირდა, ამიტომ პაკეტი მსუბუქია და ლოგიკა გასაგებია.
სერვერი (არასავალდებულო)	**Node.js სერვერი** (`npm run server`), რომელიც იგივე API ლოგიკას გაუშვებს, რასაც mock; დამატებით scrypt ჰეშირება, ხელმოწერილი HS256 JWT, CORS, ზომისა და სიხშირის ლიმიტები. Frontend-ი მას იყენებს, როცა `VITE_API_URL` არის მითითებული.
Routing და უსაფრთხოება	**React Router** + `ProtectedRoute` (RBAC), JWT-ის ფორმის ტოკენი, ვადით.
სამუშაო ინსტრუმენტები	VS Code, Git და GitHub, Vercel (დეპლოი), Chrome DevTools (რესპონსივობა, 375px), Notion (დოკუმენტაცია), GitHub Projects (დაფა), AI-ასისტენტი (Claude) როგორც კოდისა და დოკუმენტაციის მოსამზადებელი ინსტრუმენტი.

1.3. პროფესიული პასუხისმგებლობები და კომუნიკაცია

როლების შეთავსება (ინდივიდუალური პროექტი). პროექტი სრულად ინდივიდუალურია, ამიტომ ყველა როლი შევითავსე: **UI/UX** (მომხმარებლის გზა, ვაიერფრეიმები, დიზაინ-ტოკენები, მდგომარეობები), **Front-end არქიტექტურა** (საქაღალდეები, კომპონენტების ხე, routing და RBAC), **State მენეჯმენტი** (Context, ჯავშნის ლოგიკა, სლოტების კონფლიქტები), **QA** (ხელით და ავტომატური ტესტირება, ბაგების აღრიცხვა) და **დოკუმენტაცია** (Tech Spec, UX, QA ანგარიში, README).

კომუნიკაცია და კოორდინაცია. ამოცანები ჩავწერე GitHub Projects დაფაზე და Notion-ის Timeline-ზე; კოდის ცვლილებები ეტაპობრივად დავაკომიტე Git-ით; ფიდბექი პროექტის ხელმძღვანელისგან მივიღე მოსამზადებელ წარდგენაზე და მის საფუძველზე გადავაკეთე ლენდინგი და ჯავშნის ნაკადი. AI-ასისტენტთან მუშაობა აღრიცხულია AI Usage Log-ში:

პროექტის არჩევანი, ფუნქციები, გადამოწმება, უკან დაბრუნება და საბოლოო გადაწყვეტილებები ჩემია. პრაქტიკაში სამუშაო ასე მიმდინარეობდა: ჩემ მიერ დასახული მოთხოვნებიდან AI-ასისტენტი ამზადებდა კოდის ვარიანტებს, ხოლო მე ვამოწმებდი მათ რეალურ მოწყობილობებზე (კომპიუტერზე და ტელეფონზე), ეკრანის სურათებით ვაფიქსირებდი ხარვეზებს, ვწყვეტდი, რა უნდა შეცვლილიყო და რა არა, და თვითონ ვაკომიტებდი და ვაქვეყნებდი საბოლოო შედეგს. მთავარი გაკვეთილი ის იყო, რომ ახალი ფუნქციები ეტაპობრივად უნდა შემეტანა და ყოველი ეტაპი ცალკე გამეტესტა: ერთი დიდი განახლება გამიფუჭდა და Git-ით დავაბრუნე.

სწავლის შედეგი 2: პროფესიული ამოცანის დამოუკიდებლად შესრულება

2.1. ვებგვერდის არქიტექტურა და მარკირება (HTML5)

საქაღალდეების მოდულური სტრუქტურა.

```
src/
├── components/   გლობალური კომპონენტები: ui/ (Button, FormField, SelectField, Tabs,
ConfirmDialog, States), DoctorCard, ProtectedRoute, ErrorBoundary...
├── context/      AuthContext (ვინ არის შესული), ToastContext (შეტყობინებები)
├── data/         seed მონაცემები და დემო ანგარიშები
├── hooks/        useAsync (ჩატვირთვა), useDebounce, useScrolled, useOnlineStatus
├── layouts/      PublicLayout (პაციენტი/სტუმარი), DashboardLayout (ექიმი, ადმინი)
├── pages/        გვერდები; doctor/ და admin/ ქვეფოლდერებით
├── services/     http.ts, mockApi.ts, authService, doctorService, appointmentService...
├── types/        TypeScript ინტერფეისები (User, Doctor, Appointment, Review...)
└── utils/        date, roles, validation (Zod სქემები)
```

გვერდი მონაცემებს პირდაპირ არ ეხება: `page → service → http.ts → API`. ამიტომ mock სერვერის რეალურით ჩანაცვლება ერთ ფაილშია.

სემანტიკური HTML5. გამოყენებულია `<header>`, `<nav>` (მთავარი და მობილური ნავიგაცია), `<main>`, `<section>` (გვერდის ლოგიკური ბლოკები, `aria-labelledby`-ით), `<article>` (ექიმის ბარათი, ვიზიტის ბარათი), `<aside>` (გვერდითა მენიუ), `<footer>`, `<form>` და `<label>`, `<table>`, `<dl>` (მახასიათებლების წყვილები), `<time>`. მოდალური ფანჯარა `role="alertdialog"` და `aria-modal` ატრიბუტებით არის აგებული Portal-ით.

Accessibility. ველებს აქვთ მიბმული ლეიბლები (`htmlFor`/`id`, ან `sr-only` ლეიბლი); ინტერაქტიულ ელემენტებზე `aria-label` (42 ადგილას), `aria-live` სიებისა და შეტყობინებებისთვის, `aria-pressed` და `aria-selected` არჩეული მდგომარეობისთვის; ძებნა აგებულია `combobox`/`listbox` პატერნით `aria-activedescendant`-ით. **კლავიატურა:** Tab, Enter, ისრები ძებნის შეთავაზებებში, Escape ფანჯრის დასახურად, ფოკუსის ჩაკეტვა ფანჯარაში და ხილული ფოკუსის ჩარჩო. ხატულები SVG-ია და დეკორატიულია (`aria-hidden`), რასტრული სურათები არ გვაქვს, ამიტომ `alt` ტექსტი არ საჭირო.

SEO. `<html lang="ka">`, ინფორმაციული `<title>` და `meta description`, Open Graph ტეგები, `theme-color`, favicon, ერთი `<h1>` ყოველ გვერდზე, სწორი სათაურების იერარქია.

2.2. ვიზუალური სტილი და რესპონსივობა (CSS3 / Flexbox / Grid)

დიზაინ-სისტემა. ყველა ფერი, ინტერვალი, რადიუსი და ჩრდილი ერთ ადგილას, `src/index.css`-ის `:root` ბლოკშია და Tailwind-ის კონფიგურაცია მათ იყენებს:

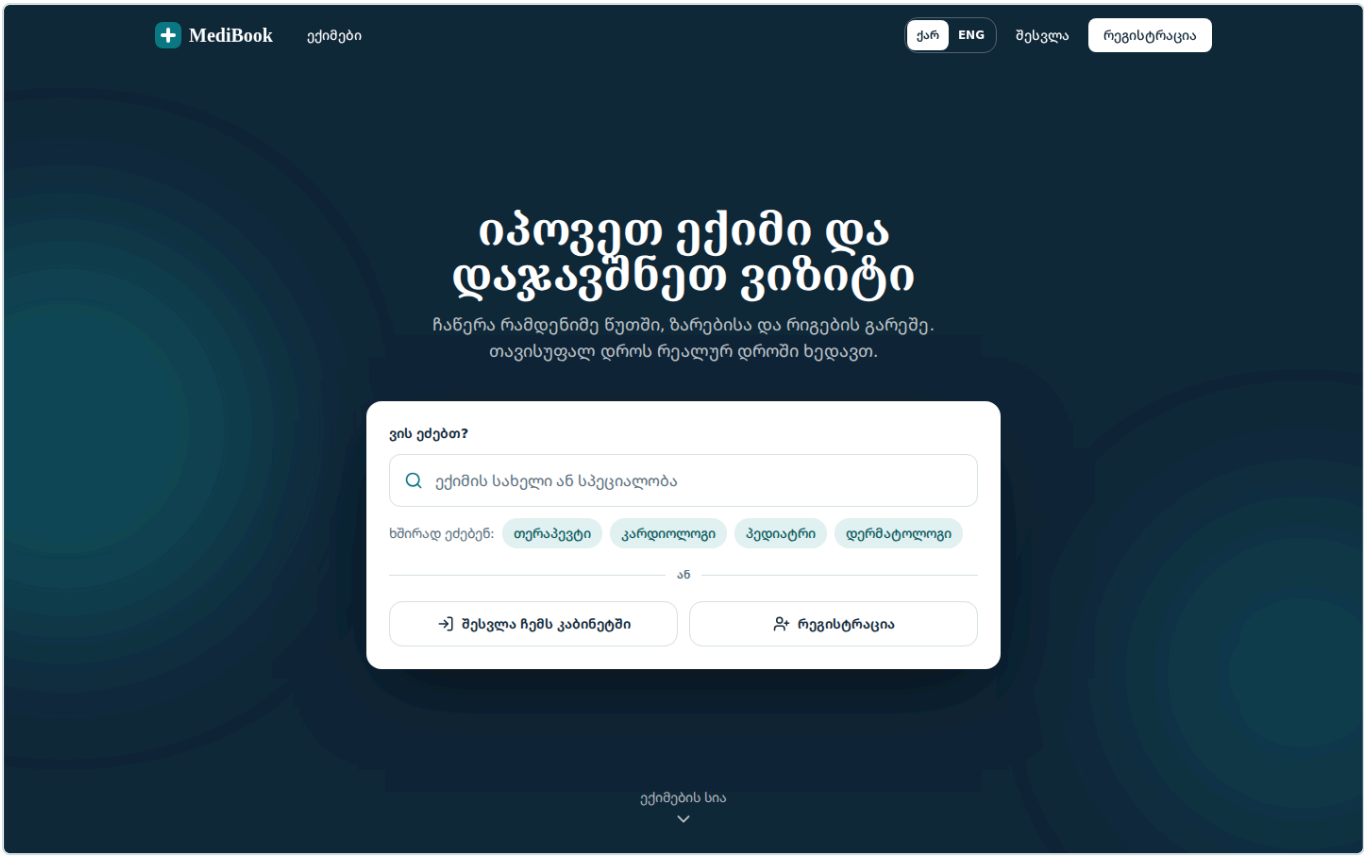
ტოკენი	მნიშვნელობა	გამოყენება
`--color-brand` / `-dark` / `-tint`	#0d7a83 / #095c63 / #e2f1f0	მთავარი ფერი, hover, მსუბუქი ფონი
`--color-ink` / `--color-muted`	#11283a / #5b6f7e	ძირითადი და მეორეხარისხოვანი ტექსტი
`--color-canvas` / `-surface` / `-line`	#f4f8f8 / #ffffff / #d9e3e6	ფონი, ბარათები, საზღვრები
`--color-danger` / `--color-ok`	#b4322f / #1f7a4d	შეცდომა და გაუქმება, წარმატება
`--space-sm..xl`	8 / 16 / 24 / 32px	8px ბადე
`--radius-md` / `-lg`, `--shadow-sm`	8px / 14px	ფორმები და ჩრდილი

ტიპოგრაფია: სათაურები Noto Serif Georgian (600/700), ტექსტი Noto Sans Georgian (400-700), ბაზური ზომა 16px, ხაზის სიმაღლე 1.6; ფონტი ქართულ ასოებს სრულად ფარავს.

Flexbox და Grid. Flexbox გამოიყენება კომპონენტების შიგნით და ერთ ხაზზე განლაგებისთვის: ნავიგაციის ზოლი (ლოგო მარცხნივ, მომხმარებელი მარჯვნივ), ბარათის ხატულა და ტექსტი, ღილაკების რიგი. **CSS Grid** გამოიყენება გვერდის მაკრო-ლეიაუთისთვის: ექიმისა და ადმინის სივრცე (`260px` გვერდითა მენიუ და `1fr` მთავარი ნაწილი), ექიმების ბარათების ბადე (1/2/3 სვეტი), სტატისტიკის ბარათები (`repeat(auto-fit, minmax(220px, 1fr))`), ექიმის გვერდის ორი სვეტი, კალენდრის 7-სვეტიანი ბადე.

რესპონსიული ადაპტაცია. განლაგება მობილურიდან იწყება (Mobile-First) და ფართოვდება წერტილებზე: `sm` 640px, `md` **768px (Tablet)**, `lg` 1024px.

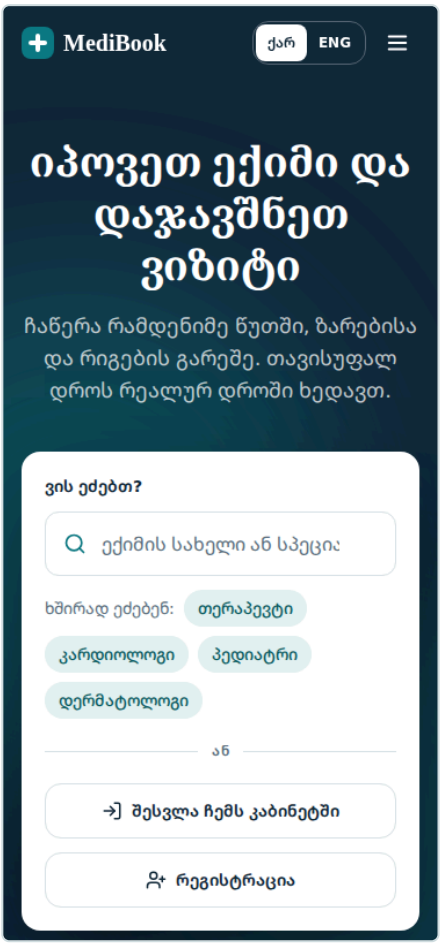
ელემენტი	Desktop 1440px	Tablet 768px	Mobile 375px
მთავარი ნავიგაცია	ჰორიზონტალური მენიუ	ჰორიზონტალური მენიუ	**ჰამბურგერ-მენიუ** (☰)
ექიმების ბარათები	3 სვეტი	2 სვეტი	1 სვეტი
ექიმის გვერდი	2 სვეტი (ინფორმაცია და დროის არჩევა)	1 სვეტი	1 სვეტი; თარიღების რიგი იხვევა საკუთარ რიგში
გვერდითა მენიუ (ექიმი/ადმინი)	ყოველთვის ჩანს (260px)	იმალება, იხსნება ☰-ით	იმალება, იხსნება ☰-ით
ცხრილები	სრული სიგანე	სრული სიგანე	იხვევა ცხრილის შიგნით, გვერდი არ ფართოვდება
პროფილის ჩანართები	ერთ რიგში	ერთ რიგში	ერთ რიგში, ერთნაირი სიმაღლით



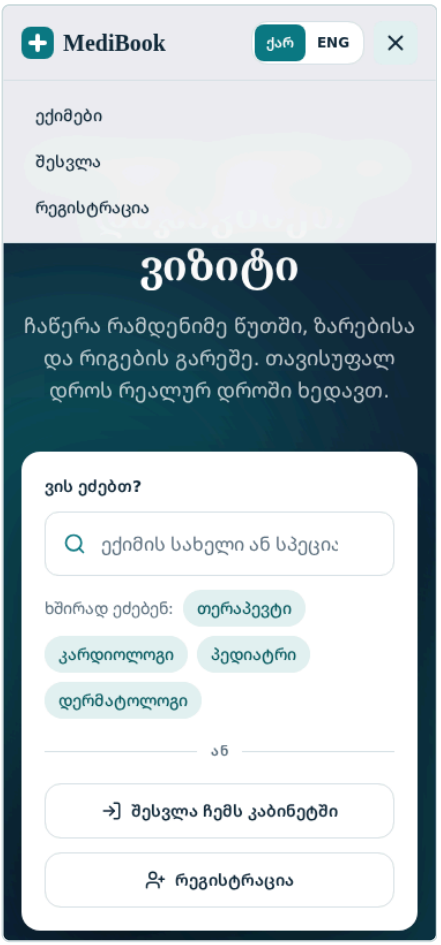
სქრინშოტი 1: Desktop 1440px (ლენდინგი)



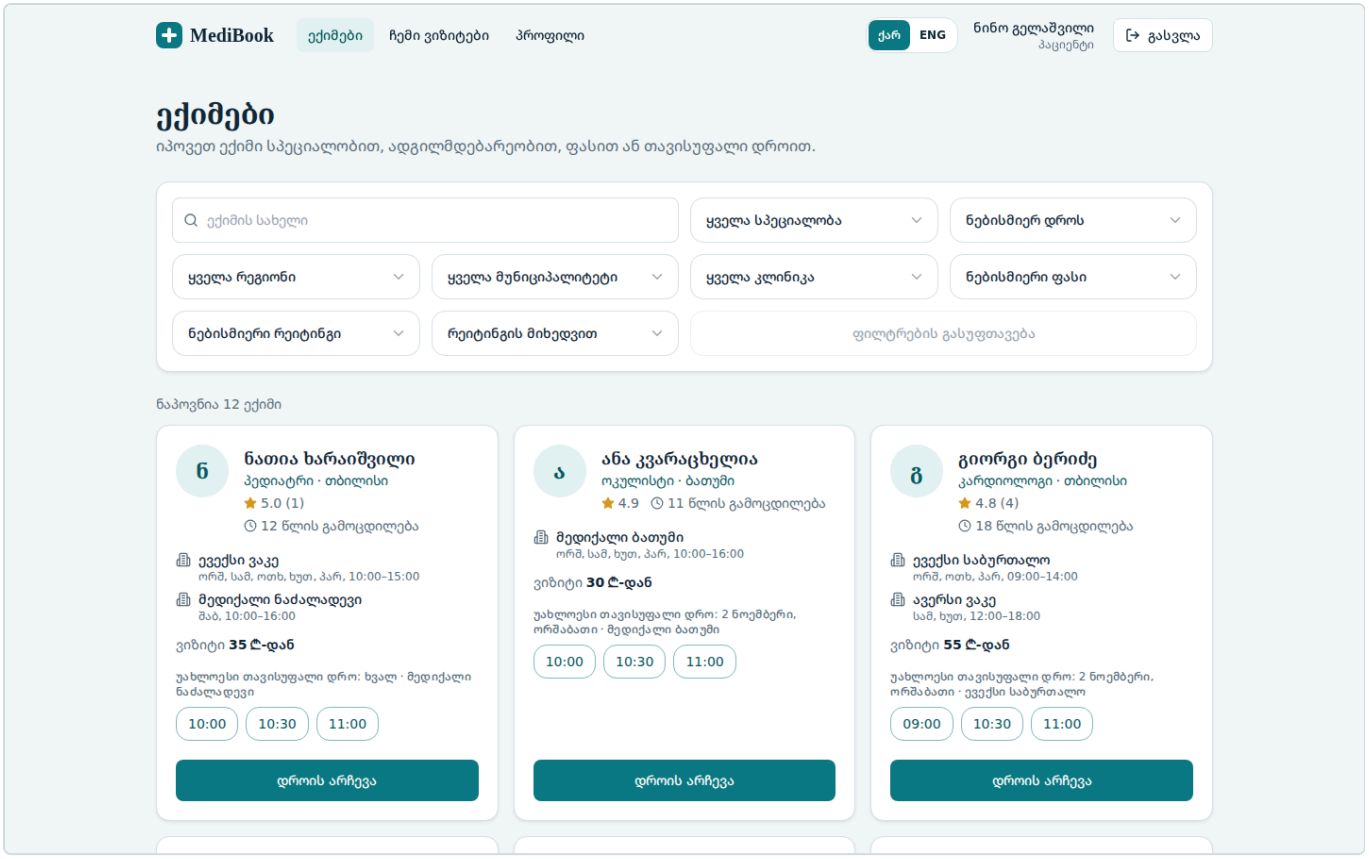
Tablet 768px



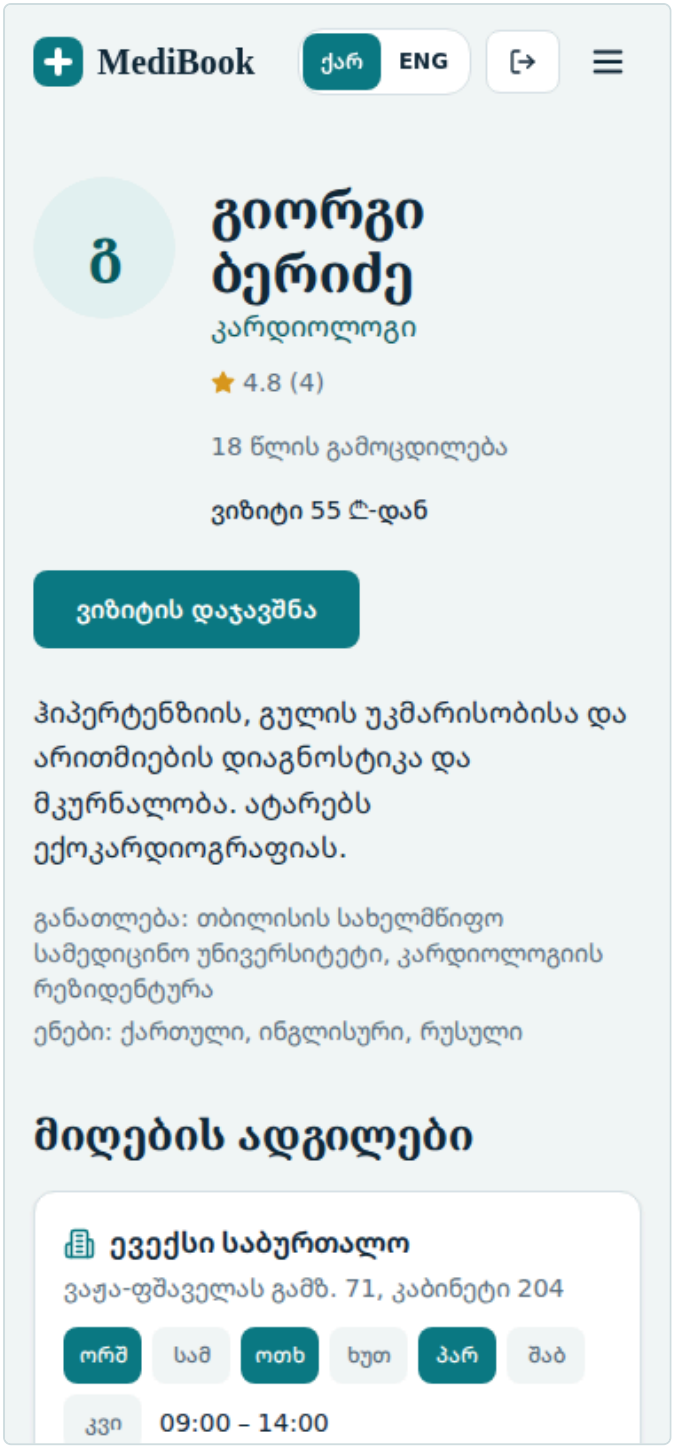
სქრინშოტი 2: Mobile 375px



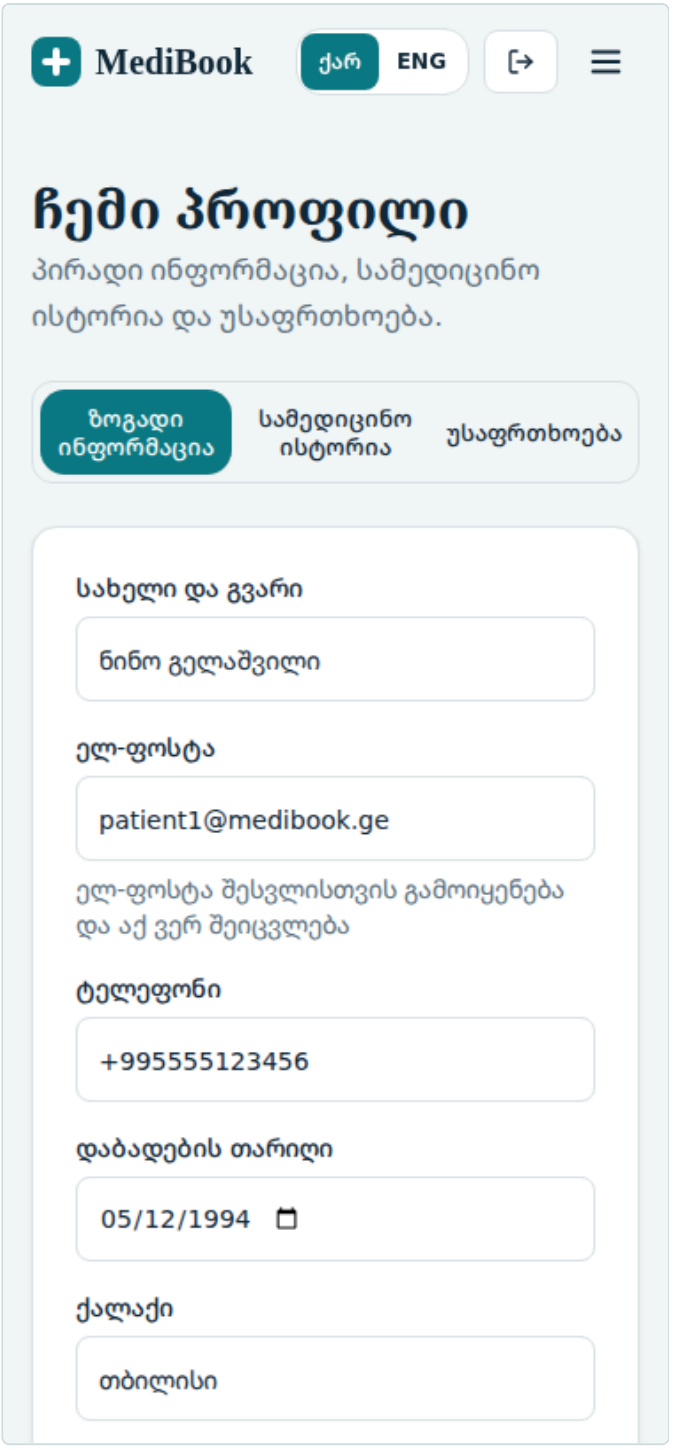
Mobile: ჰამბურგერ-მენიუ



ექიმების ძებნა, 1440px



ექიმის გვერდი, 375px



პროფილი, 375px (ჩანართები ერთ რიგში)

2.3. ინტერაქტიული ელემენტები და ფუნქციონალი (JavaScript / React)

CRUD ოპერაციები და ბიზნეს-ლოგიკა.

ოპერაცია	რა ხდება
Create	პაციენტის რეგისტრაცია; ვიზიტის დაჯავშნა კლინიკისა და ვიზიტის ტიპის არჩევით (სლოტის შემოწმებით); გადახდა (სატესტო); დიაგნოზისა და დანიშნულების ჩაწერა; შეფასება; სამედიცინო ისტორიის შევსება
Read	ექიმების სია და ძებნა; ექიმის გვერდი და თავისუფალი სლოტები; ჩემი ვიზიტები და ვიზიტის სრული ჩანაწერი; დეშბორდები, კალენდარი, პაციენტების ბარათები, სტატისტიკა
Update	პროფილი და პაროლი; ექიმის პროფესიული ინფორმაცია; დიაგნოზის რედაქტირება; ექიმის ან ადმინის მიერ გრაფიკი რამდენიმე კლინიკაში (დღეები, საათები, კაბინეტი) და ფასები; სამედიცინო ისტორია
Delete	ვიზიტის გაუქმება (პაციენტი და ადმინი), დადასტურების ფანჯრით. სტატუსი იცვლება `cancelled`-ზე და ჩანაწერი ისტორიისთვის რჩება (soft delete); ჩანართიდან სიის ელემენტების წაშლა სამედიცინო ისტორიაში; ექიმის დეაქტივაცია

State Management და კომპონენტთაშორისი კავშირი. გლობალური მდგომარეობა ორ Context-შია: `AuthContext` (მიმდინარე მომხმარებელი, შესვლა/გასვლა, `updateUser`) და `ToastContext` (შეტყობინებები). მაგალითად, როცა მომხმარებელი პროფილში სახელს ცვლის, ზედა ნავიგაციის სახელი იმავე წამს განახლდება (Header Counter-ის ანალოგიური კავშირი). გვერდების ლოკალური მდგომარეობა `useAsync` ჰუკშია (ჩატვირთვა, შეცდომა, განახლება).

localStorage-თან სინქრონიზაცია. მთელი ბაზა ინახება `localStorage`-ში, სესიის ტოკენი `sessionStorage`-ში. როცა ერთ ბრაუზერის ჩანართში მონაცემი იცვლება, მეორე ჩანართი `storage` მოვლენით ჩუმად განახლდება: პირველი პაციენტის დაჯავშნილი დრო მეორეს გადატვირთვის გარეშე უჩანს როგორც დაკავებული.

დინამიური ძიება და ფილტრაცია. ძებნა ეძებს სახელით და სპეციალობით; შეყვანისას ჩნდება შეთავაზებები (ექიმები და სპეციალობები); ფილტრები: სპეციალობა, კლინიკა, რეიტინგი, დალაგება; ყველა პარამეტრი ინახება მისამართში (`?specialty=...`), ამიტომ ბმული გასაზიარებელია; `useDebounce` 300ms.

ექიმი რამდენიმე კლინიკაში და ფასები. ექიმს აქვს რამდენიმე ლოკაცია, თითოეულს საკუთარი დღეებით, საათებით და კაბინეტით. ექიმის გვერდზე ჩანს, სად და როდის იღებს და რა ღირს თითოეული ვიზიტი; თავისუფალი დროები კლინიკების მიხედვით არის დაჯგუფებული. სისტემა ამოწმებს, რომ ერთი ექიმის საათები სხვადასხვა კლინიკაში ერთ დღეს არ იკვეთებოდეს და რომ უკვე დაჯავშნილი ვიზიტები ახალ გრაფიკში ჯდებოდეს.

გადახდა (სატესტო). ჯავშნისას პაციენტი ირჩევს ონლაინ გადახდას ან რეგისტრატურაში გადახდას. ონლაინ არჩევისას გადადის გადახდის გვერდზე (ბარათის ფორმა ვალიდაციით, Apple Pay-ის სატესტო ფანჯარა); გვერდი რეალურ ბანკს არ უკავშირდება. წარმატების შემდეგ ჩნდება ქვითარი. გაუქმებისას გადახდილი თანხა „უბრუნდება“ სტატუსში გადადის.

ძიება და ფილტრები დიდი ისტორიისთვის. პაციენტის, ექიმისა და ადმინის ვიზიტების სია ფილტრდება სერვერის მხარეს: ტექსტი, სტატუსი, თარიღების შუალედი, კლინიკა, სპეციალობა, ექიმი, გადახდა, დალაგება, გვერდებად დაყოფა. ადმინისტრატორს სამედიცინო ტექსტი არც ჩანს და არც ძიებაში მონაწილეობს. ეს ერთი საერთო `VisitHistory` კომპონენტია სამივე როლისთვის.

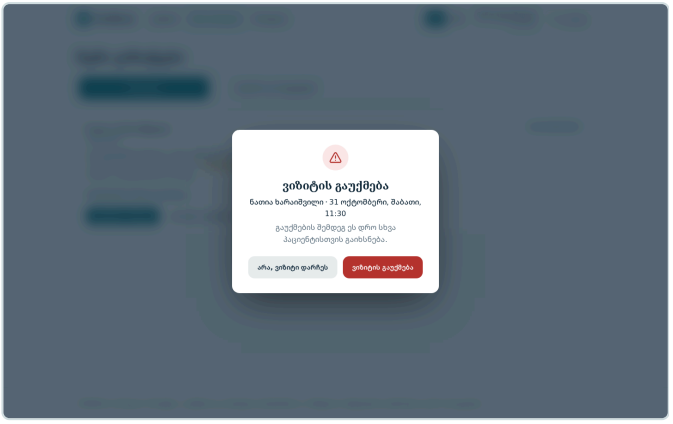
ორი ენა (ქართული და ინგლისური). ზედა ზოლში გადამრთველი **ქარ | ENG** ყველა როლისთვის. 770 ტექსტიდან თითოეული გატარებულია `tr()` ფუნქციაში; API-დან მოსული

სახელები, კლინიკები და სპეციალობები ინგლისურ რეჟიმში ავტომატურად ითარგმნება, თარიღი და მრავლობითი რიცხვი ენის წესით იწერება, ხოლო `<html lang>` ენასთან ერთად იცვლება. ტესტით დადასტურებულია, რომ ინგლისურ რეჟიმში გვერდებზე ქართული ტექსტი აღარ რჩება.

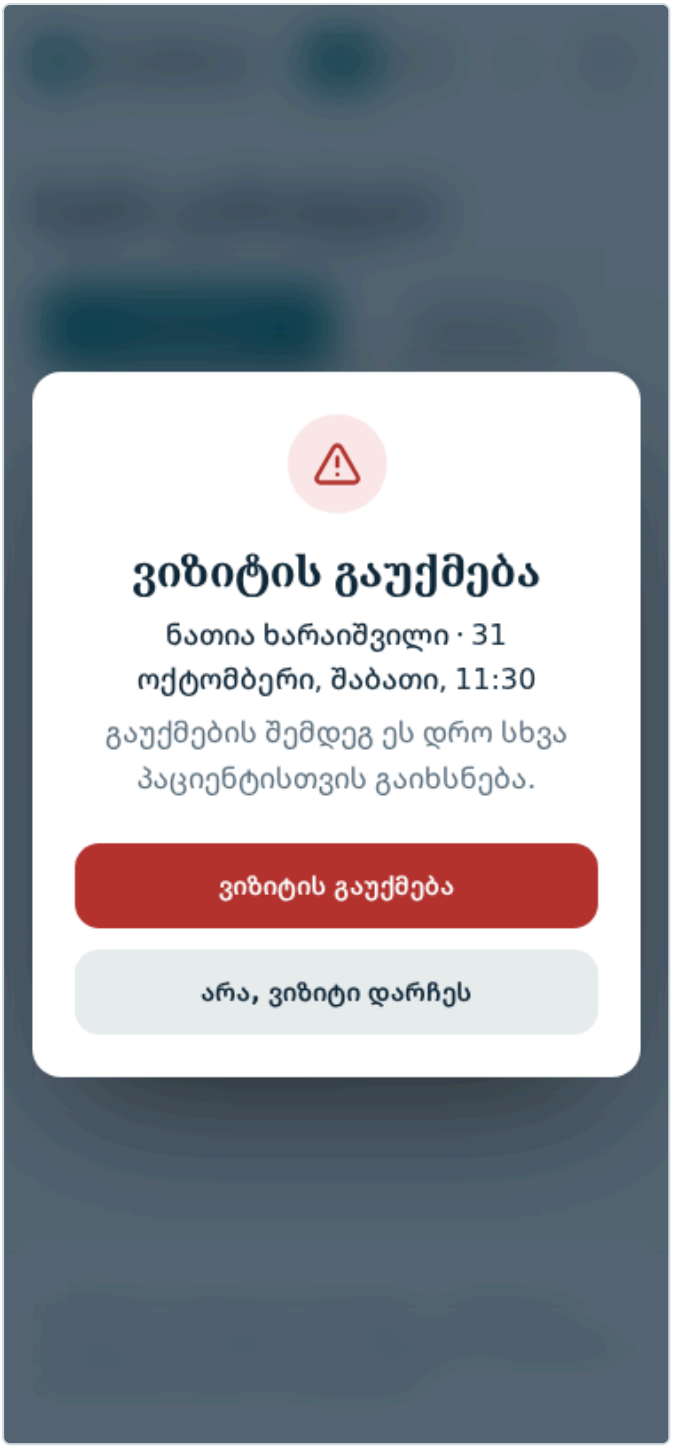
თვის კალენდარი. ვიზიტის დაჯავშნა შესაძლებელია დღევანდელი დღიდან 30 დღით ადრე: კალენდარში ექიმის არამომუშავე დღეები მქრქალია, ხოლო თუ პირველი დღე სავსეა, არჩევანი თავისით გადადის უახლოეს დღეზე, სადაც თავისუფალი დროა.

მოდალური ფანჯრები. ვიზიტის გაუქმების დადასტურება არის ერთიანი `ConfirmDialog` კომპონენტი, რომელიც `ReactDOM.createPortal`-ით ეკრანის ცენტრში ჩნდება; ფონი მქრქალდება, გვერდი არ გადაიხვევა, ფოკუსი ჩაკეტილია ფანჯარაში და პირველად უსაფრთხო ღილაკზეა; Escape, ფანჯრის გარეთ დაჭერა და ნაცრისფერი ღილაკი ვიზიტს უცვლელს ტოვებს. იგივე ფანჯარა მუშაობს პაციენტთანაც და ადმინთანაც.

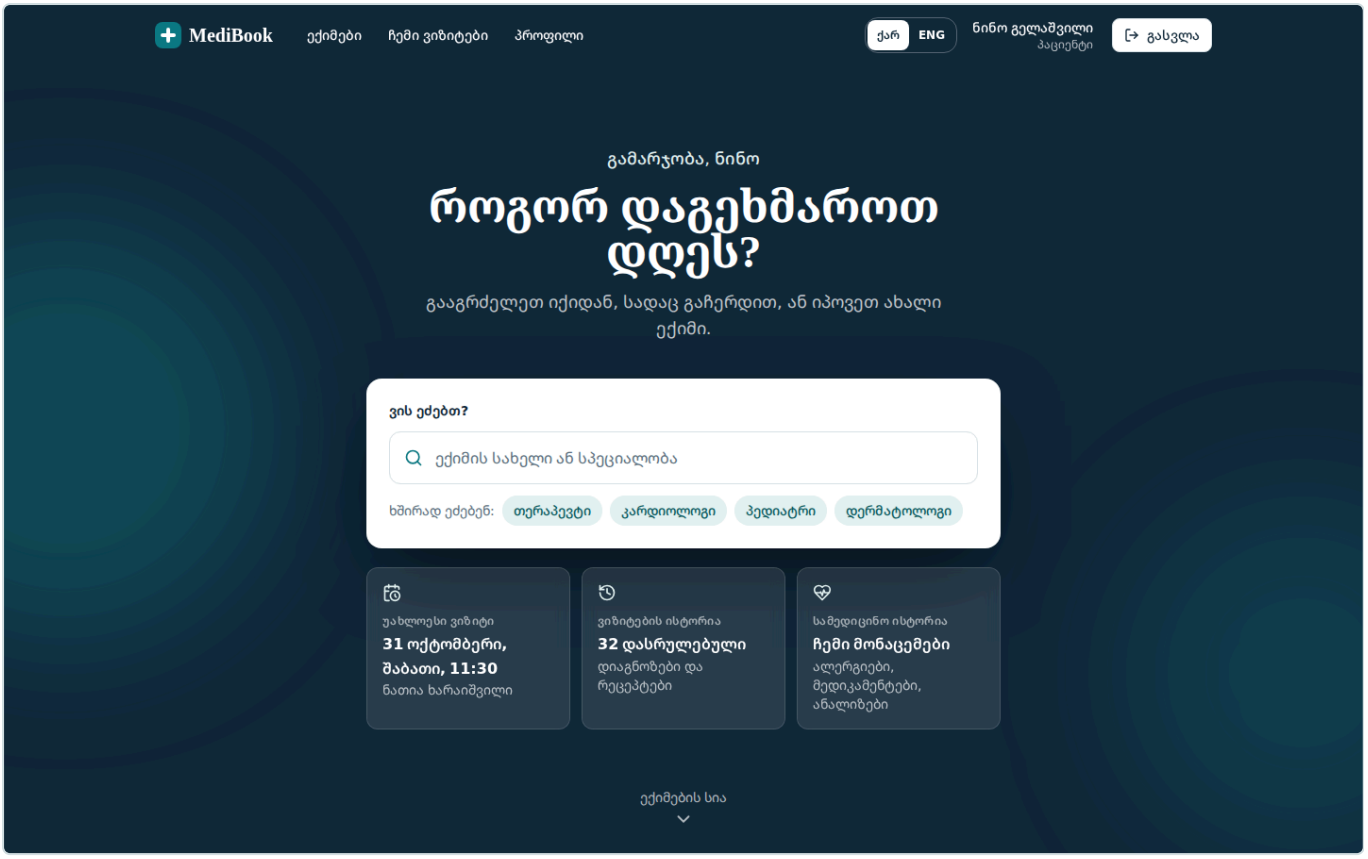
ფორმის ვალიდაცია. React Hook Form + Zod, შეტყობინება ველის დატოვებისას (`onTouched`): სახელი მინიმუმ 2 სიმბოლო; ელ-ფოსტის ფორმატი; ტელეფონი `+995` და 9 ციფრი; პაროლი მინიმუმ 8 სიმბოლო, ციფრი და სპეციალური სიმბოლო; პაროლების დამთხვევა; ლიმიტები ტექსტურ ველებზე. იგივე სქემები სერვერის მხარესაც მოწმდება.



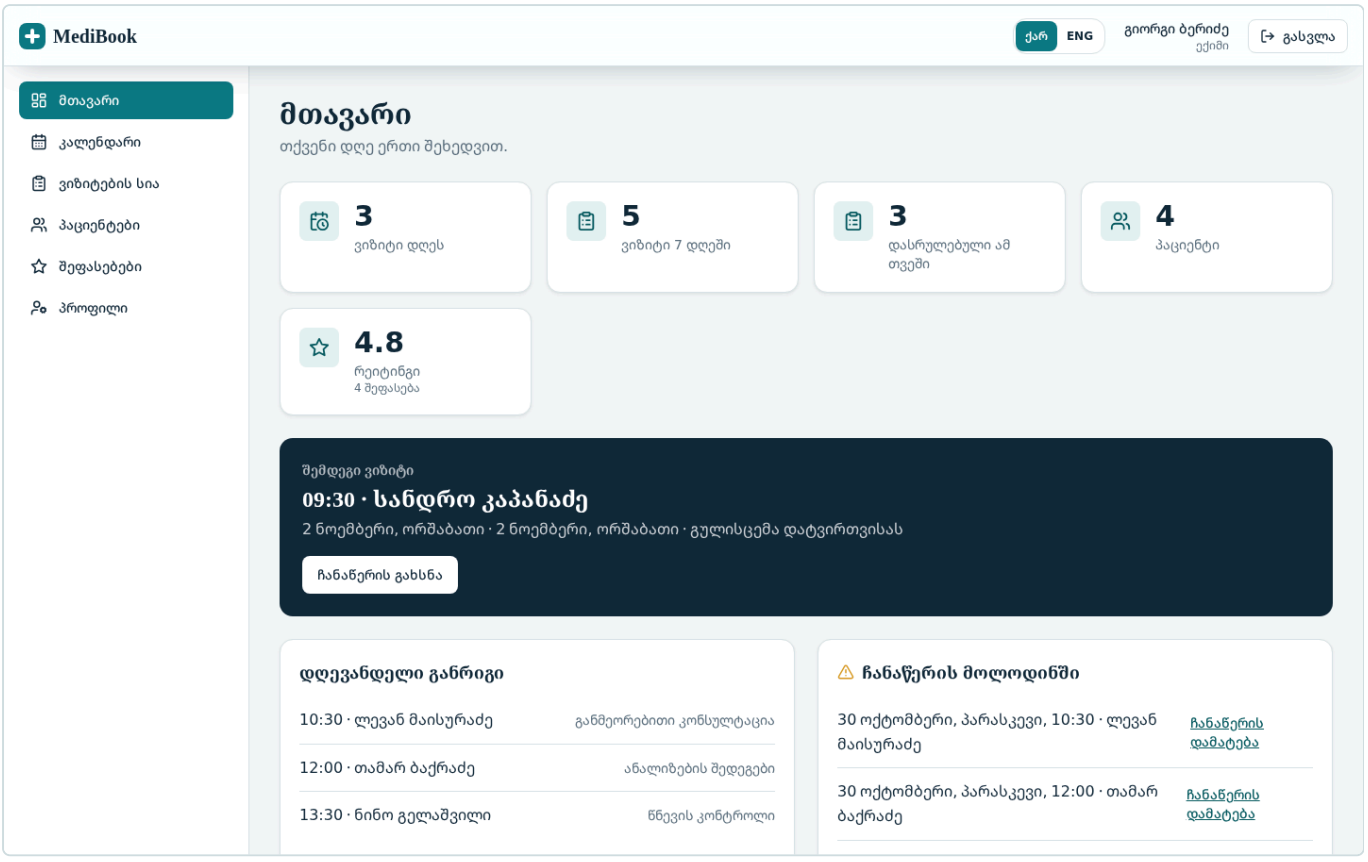
იგივე ფანჯარა 1440px-ზე



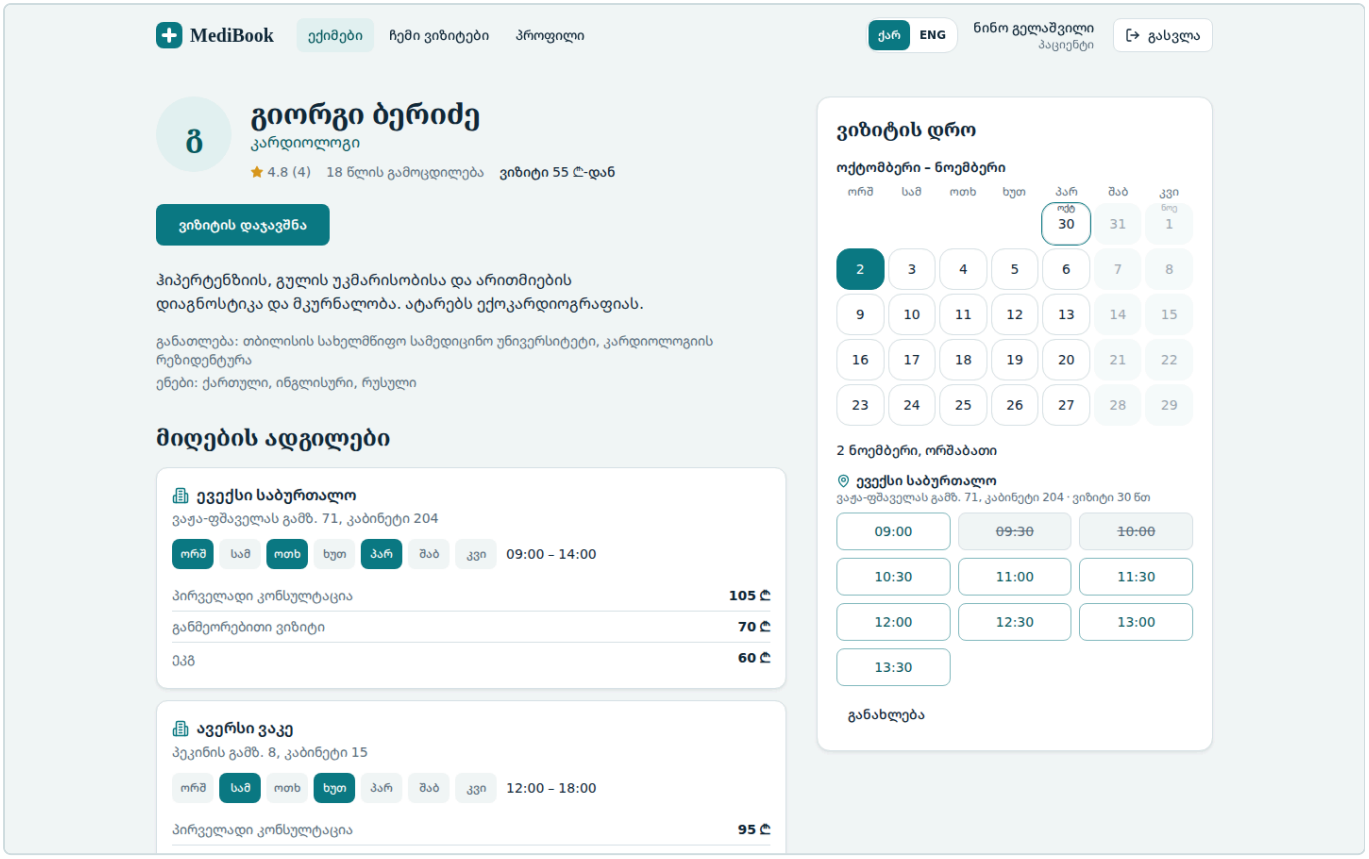
სქრინშოტი 3: მოდალური ფანჯარა (ვიზიტის გაუქმება), 375px



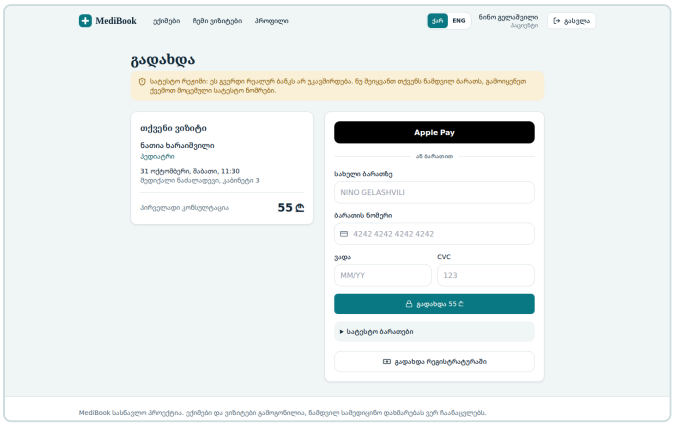
პაციენტის მთავარი გვერდი შესვლის შემდეგ



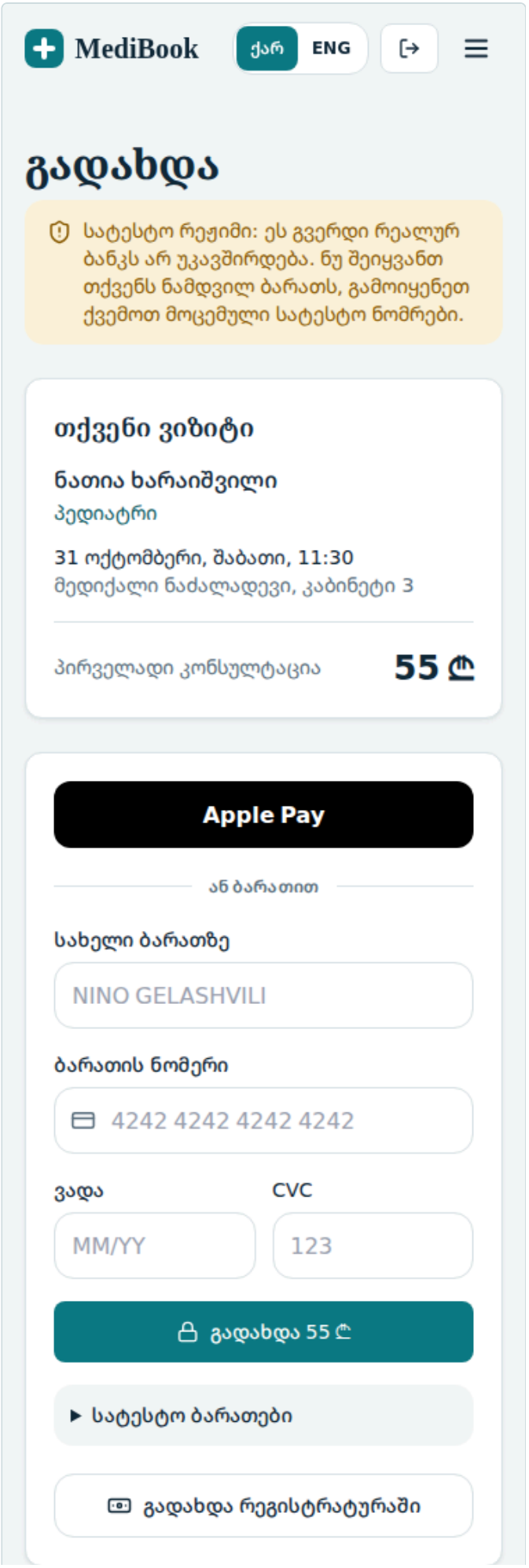
ექიმის მთავარი გვერდი

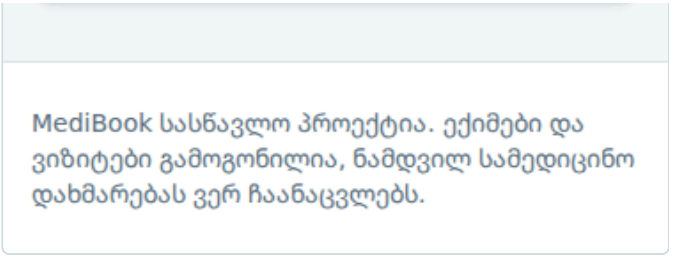


ექიმის გვერდი: რამდენიმე კლინიკა, ფასები, დრო კლინიკების მიხედვით

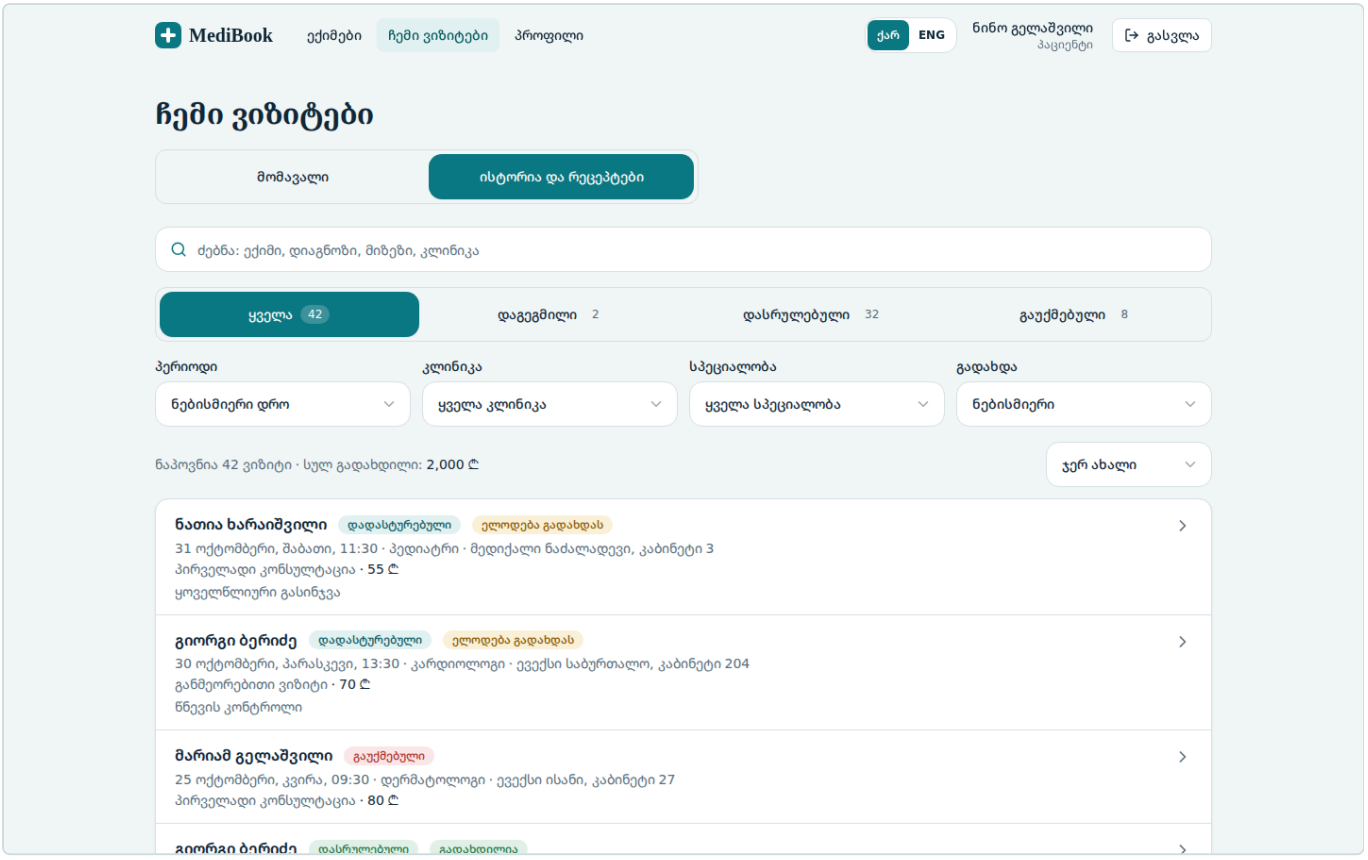


სატესტო გადახდის გვერდი, 1440px

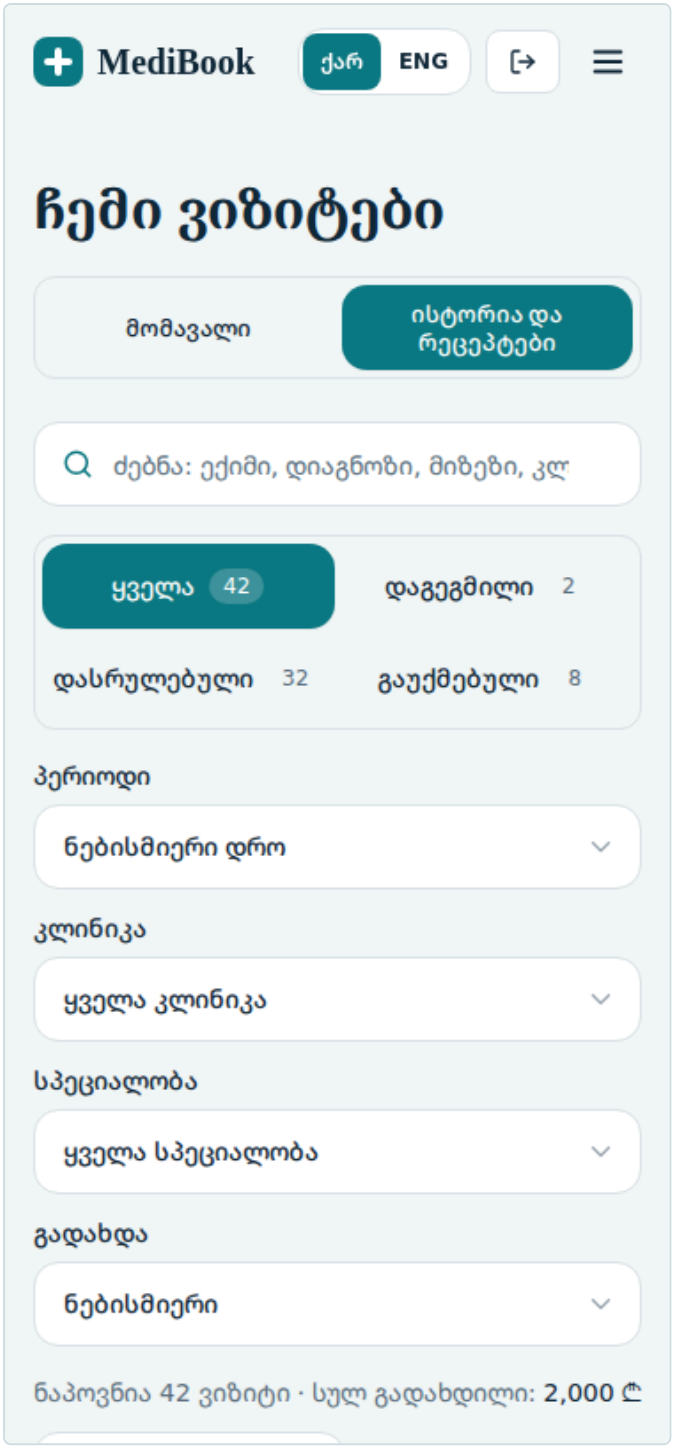

MediBook
ქარ
ENG
გადახდა
სატესტო რეჟიმი: ეს გვერდი რეალურ ბანკს არ უკავშირდება. ნუ შეიყვანთ თქვენს ნამდვილ ბარათს, გამოიყენეთ ქვემოთ მოცემული სატესტო ნომრები.
თქვენი ვიზიტი
ნათია ხარაიშვილი
პედიატრი
31 ოქტომბერი, შაბათი, 11:30
მედიქალი ნაძალადევი, კაბინეტი 3
პირველადი კონსულტაცია
55 ₾
Apple Pay
ან ბარათით
სახელი ბარათზე
NINO GELASHVILI
ბარათის ნომერი
4242 4242 4242 4242
ვადა
CVC
MM/YY
123
გადახდა 55 ₾
▶ სატესტო ბარათები
გადახდა რეგისტრატურაში



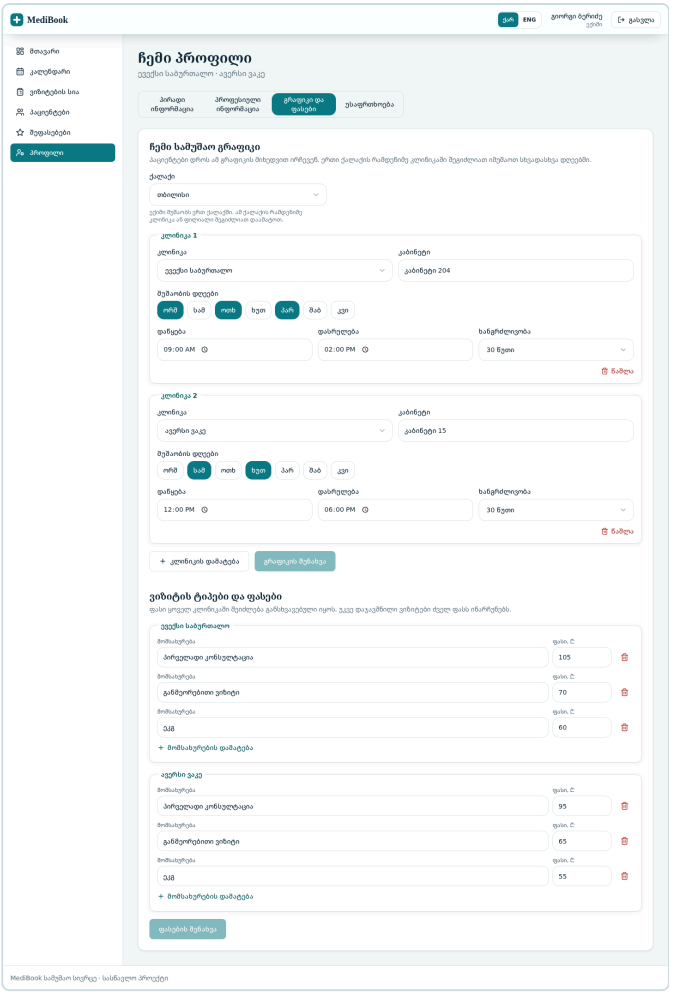
იგივე გვერდი, 375px



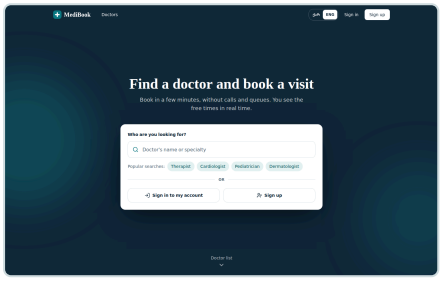
ვიზიტების ისტორია: ძებნა, სტატუსის ჩანართები, ფილტრები, 1440px



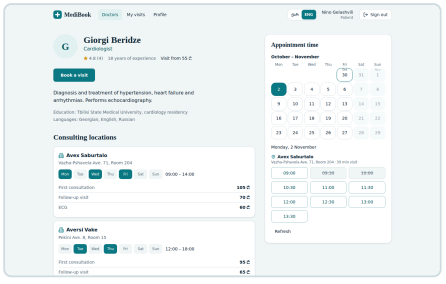
ვიზიტების ისტორია, 375px



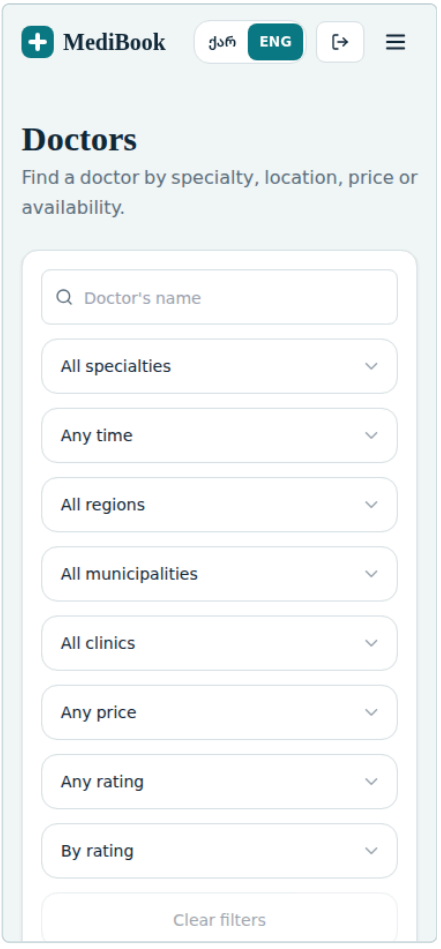
ექიმის გრაფიკი და ფასები



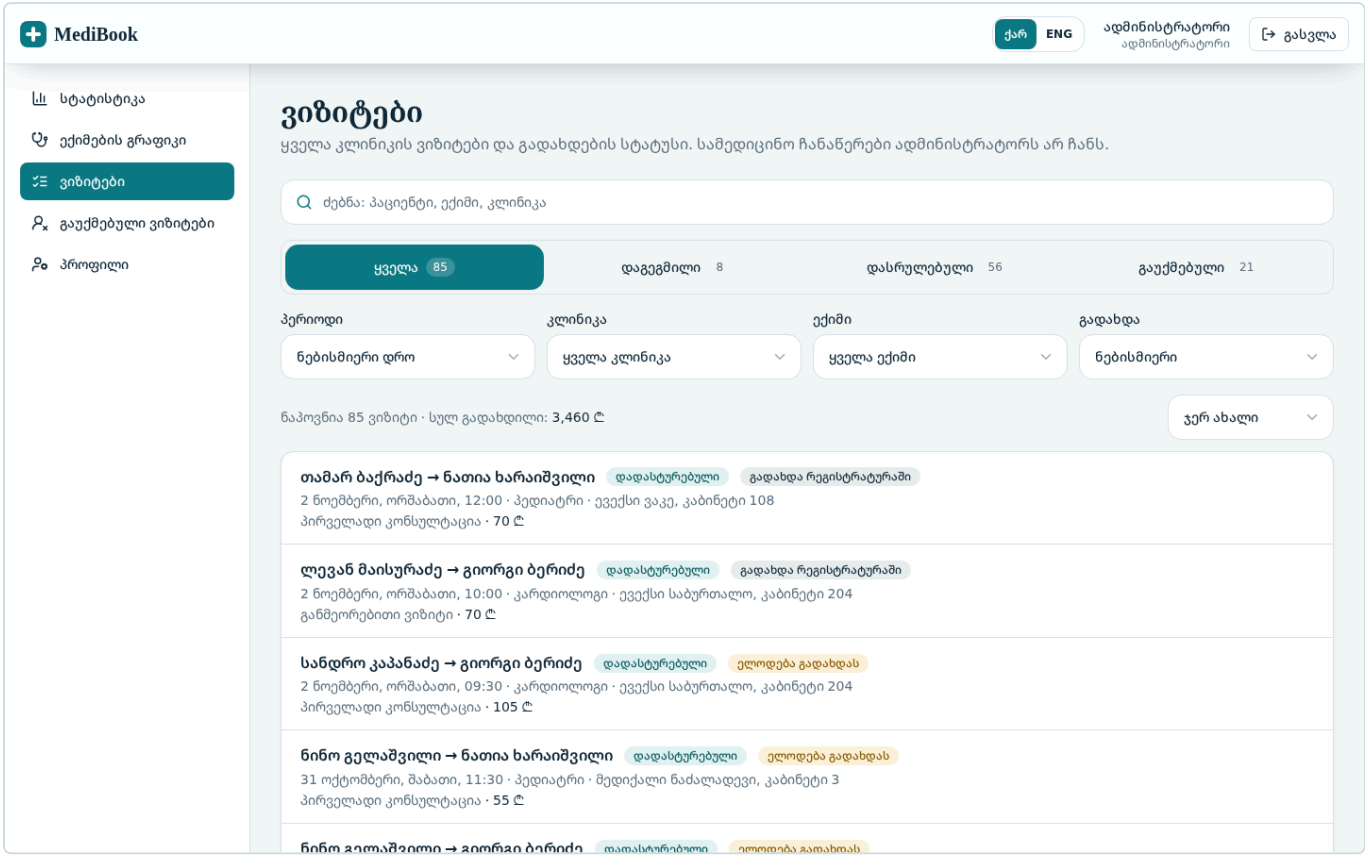
ინგლისური ვერსია: მთავარი გვერდი



ინგლისური ვერსია: ექიმის გვერდი



ინგლისური ვერსია, 375px



ადმინისტრატორის ვიზიტები ფილტრებით

2.4. ვერსიების კონტროლი (Git / GitHub)

რეპოზიტორიის ჰიგიენა. `.gitignore`-ში გაწერილია `node_modules`, `dist`, `.env` და `.env.*` (გარდა `.env.example`-ისა), `*.local`, ლოგები და სისტემური ფაილები. საიდუმლო მონაცემები რეპოზიტორიაში არ ხვდება.

ბრენჩები. სამუშაო მიმდინარეობდა `main` ბრენჩზე, ყოველი ეტაპი ცალკე კომიტით და უკან დაბრუნების შესაძლებლობით (გამოვიყენე `git revert` ერთი უშედეგო განახლების გასაუქმებლად). ახალი ფუნქციებისთვის (Version 2.0) იქმნება ცალკე ბრენჩი. ვერსიების გაერთიანების პროცესის საჩვენებლად შევქმენი ბრენჩი `docs/project-links`, ავტვირთე GitHub-ზე და `main`-ში გავაერთიანე Pull Request #1-ით (კომიტი `d8e951d`).

სემანტიკური კომიტების მაგალითები:

```
8a57263  feat: landing page, welcome cabinet, booking flow, cancel dialog, tab and select
polish
e3a2be9  fix: doctor page overflow on mobile, update README
481aff6  docs: add project links                      (ბრენჩი docs/project-links)
d8e951d  Merge pull request #1 from tinatinmaisurradze/docs/project-links
```

სრული ისტორია ჩანს GitHub-ზე; კომიტები იწერება სემანტიკური პრეფიქსებით: `feat` (ახალი ფუნქცია), `fix` (შესწორება), `docs` (დოკუმენტაცია).

2.5. ოპტიმიზაცია და პრობლემების გადაჭრა

წარმადობა.

- `useDebounce` (300ms): ექიმების ძებნა მხოლოდ მაშინ აგზავნის მოთხოვნას, როცა მომხმარებელმა წერა შეწყვიტა.
- `useMemo` (დღეების სია, კალენდრის მონაცემები, ფილტრაცია), `useCallback` (`useAsync` ჰუკი), `React.memo` (`DoctorCard`, `Avatar`, `Rating`, `StatusBadge`, `RatingSummary`), რომ უცვლელი ბარათები ხელახლა არ დაიხატოს.
- `React.lazy` + `Suspense`: მძიმე და როლისთვის სპეციფიკური გვერდები ჩაიტვირთება საჭიროებისას (კოდის გაყოფა), ამიტომ პირველი ჩატვირთვა მსუბუქია.
- მედია: ყველა ხატულა SVG-ია (Lucide), რასტრული სურათები არ გვაქვს, ამიტომ WebP და `loading="lazy"` არ საჭიროებს; ფონტები `preconnect`-ით და `display=swap`-ით იტვირთება.

სისტემის მდგრადობა (Defensive Programming).

- **ErrorBoundary:** გვერდზე მოულოდნელი შეცდომა თეთრ ეკრანს არ იძლევა: ჩანს მეგობრული შეტყობინება ღილაკებით „გვერდის განახლება“ და „მთავარ გვერდზე“; ნავიგაციისას ბარიერი იწმინდება. ერთი ბარიერი მარშრუტებზეა, მეორე მთელ აპლიკაციაზე.
- **Double-Click დაცვა:** ფორმის გაგზავნისას ღილაკი გამოირთვება და ჩანს ბრუნვადი ხატულა (`isSubmitting` / `loading`); სერვერი დამატებით თავიდან ამოწმებს სლოტს.
- **Offline / Fallback:** კავშირის გაქრობისას ეკრანის ქვემოთ ჩნდება შეტყობინება; აპლიკაცია მუშაობს ამ მოწყობილობაზე შენახულ მონაცემებზე. ყველა სიას აქვს Loading (Skeleton), Empty და Error (ღილაკით „თავიდან ცდა“) მდგომარეობა.
- ბრუტ-ფორსის დაცვა: 5 წარუმატებელი შესვლის შემდეგ ანგარიში 5 წუთით იბლოკება.
- **უსაფრთხოება (რეალური სერვერის რეჟიმში):** პაროლები scrypt-ით ჰეშირებულია; ტოკენი ხელმოწერილი HS256 JWT-ია (შეცვლილი ან გაყალბებული ტოკენი უარიყოფა,

დამოწმებულია ტესტით); CORS მხოლოდ დაშვებულ მისამართებზე; მოთხოვნის ზომისა და სიხშირის ლიმიტი; მონაცემები არასოდეს ილოგება.

- **HTTP სათაურები:** `vercel.json`-ში Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, HSTS, Permissions-Policy; აპლიკაცია გადამოწმებულია ბრაუზერში და პოლიტიკის დარღვევა არ ფიქსირდება.
- **დამოკიდებულებების აუდიტი:** `npm audit` აჩვენებს ორ ცნობილ საკითხს (react-router, zod), რომლებიც ამ აპლიკაციაში ექსპლუატირებადი არ არის (ბმულებს მომხმარებლის მიერ მოცემულ მისამართზე არ ვაგზავნით, სერვერი მოთხოვნის ზომას ზღუდავს); განახლება zod 4-ზე შეტანილია Roadmap-ში.

გადაჭრილი ტექნიკური პრობლემები.

ქეისი 1: გამჭვირვალე ნავიგაციის ზოლი. გადასქროლისას გვერდის ტექსტი ზედა ზოლს ეფარებოდა. მიზეზი: Tailwind-ში გამჭვირვალობის კლასი (`bg-surface/95`) CSS ცვლადთან არ მუშაობს და ფონი საერთოდ არ იქმნებოდა (იგივე პრობლემა ფოკუსის ჩარჩოსა და გამორთულ ღილაკზეც იყო). გამოსავალი: ცალკე კლასი `.app-header` (`rgba` ფონი, `backdrop-filter: blur`, ჩრდილი) და ფერის კონკრეტული მნიშვნელობები სხვა ადგილებში; ზედა ზოლი გადასქროლამდე გამჭვირვალეა, შემდეგ მქრქალი თეთრი.

ქეისი 2: ექიმის გვერდი მობილურზე იჭრებოდა. 375px სიგანეზე ყველა გვერდის შემოწმებისას ექიმის გვერდი მარჯვნივ 571px-ით სცდებოდა. მიზეზი: ბადის სვეტს მინიმალური სიგანე არ ჰქონდა და თარიღების გრძელი რიგი მთელ გვერდს აფართოებდა. გამოსავალი: `grid-template-columns: minmax(0, 1fr)` და `min-w-0`, რის შემდეგაც თარიღების რიგი იხვევა საკუთარ რიგში.

ქეისი 3 (პროცესი): დიდი განახლების გაუქმება. ერთდროულად ბევრი ფაილის განახლებამ ჩემს კომპიუტერზე ვიზუალი დააზიანა. ვერსია `git revert`-ით დავაბრუნე და ცვლილებები გავყავი მცირე, ცალკე გატესტილ ეტაპებად.

სწავლის შედეგი 3: შესრულებული ამოცანების ანგარიშგება

3.1. პროექტის ტექნიკური დოკუმენტაცია (README.md)

დადასტურებულია, რომ რეპოზიტორიაში განთავსებულია სრულყოფილი `README.md`: პროექტის აღწერა და ბმულები, ფუნქციები როლების მიხედვით, ტექნოლოგიური სტეკის ცხრილი დასაბუთებით, საქაღალდეების სტრუქტურა, გაშვების ინსტრუქცია, სატესტო ანგარიშები, უსაფრთხოების და მდგრადობის აღწერა, ეკრანები და Roadmap.

```
git clone https://github.com/tinatinmaisurradze/medibook.git
cd medibook
npm install
npm run dev        # http://localhost:5173
```

როლი	ელ-ფოსტა (პაროლი: Demo1234!)
პაციენტები	patient1@medibook.ge, patient2@medibook.ge, patient3@medibook.ge
ექიმები	doctor1@medibook.ge, doctor2@medibook.ge, doctor3@medibook.ge
ადმინისტრატორი	admin@medibook.ge

3.2. პრეზენტაცია და პროდუქტის წარდგენა

სადემონსტრაციო სცენარი (Happy Path).

1. **კლინიკის არჩევა და გადახდა:** ექიმის გვერდზე აირჩიე დრო კლინიკის ქვეშ, ჯავშნისას ვიზიტის ტიპი და „გადახდა ონლაინ“, გადახდის გვერდზე სატესტო ბარათი ან Apple Pay (დემო), მიიღე ქვითარი.
2. **ენა:** დააჭირე **ENG**, გვერდი ინგლისურად გადაირთვება; დაბრუნდი **ქარ**-ზე.
3. **ლენდინგი:** გახსენი საიტი და აჩვენე საძიებო ბარათი და ორი გზა: ძებნა ან შესვლა კაბინეტში.
4. **ძებნა:** ჩაწერე ექიმის სახელი, აირჩიე შეთავაზებიდან ან დააჭირე Enter, გადადის ექიმის გვერდზე.
5. **ჯავშნის დაწყება სტუმრად:** დააჭირე „ვიზიტის დაჯავშნა“, საიტი შესვლის გვერდზე გადადის განმარტებით.
6. **შესვლა:** დემო ანგარიშზე ერთი დაწკაპუნებით შედი (პაციენტი 1), ბრუნდები იმავე ექიმთან დროის არჩევაზე.
7. **ორი პაციენტი:** სხვა ჩანართში შედი პაციენტი 2-ით და გახსენი იგივე ექიმი; პირველით დაჯავშნე დრო, მეორე ჩანართში ის გადადის „დაკავებულია“-ზე გადატვირთვის გარეშე.
8. **გაუქმება:** „ჩემი ვიზიტები“, ვიზიტის გაუქმების ფანჯარა, „არა“ და მერე დადასტურება.
9. **ექიმი:** შედი ექიმით, აჩვენე მთავარი გვერდი, კალენდარი და ვიზიტზე დიაგნოზის ჩაწერა.
10. **პაციენტი:** ვიზიტის ჩანაწერიში ჩანს დიაგნოზი და დანიშნულება, ექიმის შეფასება.
11. **ადმინისტრატორი:** სტატისტიკა და გრაფიკი; სამედიცინო ტექსტი არ ჩანს.
12. **შედეგი:** ერთი სისტემა სამი როლისთვის, ორმაგი ჯავშნის გარეშე და დაცული სამედიცინო მონაცემებით.

შეჯამება. შეიქმნა ვებ-აპლიკაცია, რომელიც ჯავშნის პროცესს პაციენტისთვის ამარტივებს, ექიმისთვის განრიგს ასისტენტურად უჩვენებს და ადმინისტრატორს გრაფიკებს მართვის საშუალებას აძლევს. პროექტი გამოქვეყნებულია და მუშაობს 375px, 768px და 1440px სიგანეზე; სერვერის წესები და ძირითადი სცენარები ავტომატურად გადამოწმებულია; ბაგები და გადაწყვეტილებები აღრიცხულია QA ანგარიშში.

Version 2.0 Roadmap.

1. **რეალური გადახდის პროვაიდერი:** სატესტო გვერდის ნაცვლად ბანკის ან გადახდის სერვისის ინტეგრაცია, ანგარიშსწორება კლინიკებთან.
2. **გამოქვეყნებული სერვერი და ბაზა (PostgreSQL) და კლინიკის ადმინისტრატორის როლი,** რომელიც მხოლოდ საკუთარ კლინიკას ხედავს (სააბონენტო მოდელი). სამედიცინო მონაცემების რეალურ გამოყენებამდე საჭიროა პერსონალური მონაცემების დაცვის მოთხოვნების შესრულება.
3. **შეხსენებები ელფოსტით/SMS-ით, ავტომატური end-to-end ტესტები** და ძებნის გაუმჯობესება ათასობით ექიმზე.